users, an attacker may be able to trick the service to performing malicious actions on their behalf.

Kubernetes critical API server vulnerability

In 2018, the Kubernetes project itself reported a critical vulnerability allowing this kind of confused deputy attack (https://rancher.com/blog/2018/2018-12-04-k8s-cve/). In the attack, a user made an initial request to the Kubernetes API server, which authenticated the request and applied access-control checks. It then made its own connection to a backend service to fulfill the request. This API request to the backend service used highly privileged Kubernetes service account credentials, providing administrator-level access to the entire cluster. The attacker could trick Kubernetes into leaving the connection open, allowing the attacker to send their own commands to the backend service using the service account. The default configuration permitted even unauthenticated users to exploit the vulnerability to execute any commands on backend servers. To make matters worse, Kubernetes audit logging filtered out all activity from system accounts so there was no trace that an attack had taken place.

You can avoid confused deputy attacks in service-to-service calls that are carried out in response to user requests by ensuring that access-control decisions made in backend services include the context of the original request. The simplest solution is for frontend services to pass along the username or other identifier of the user that made the original request. The backend service can then make an access-control decision based on the identity of this user rather than solely on the identity of the calling service. Service-to-service authentication is used to establish that the request comes from a trusted source (the frontend service), and permission to perform the action is determined based on the identity of the user indicated in the request.

> **TIP** As you'll recall from chapter 9, capability-based security can be used to systematically eliminate